Top MySQL & SQL Interview Questions & Answers

A Complete Guide for Interview Preparation

Including Scenario-Based Questions & Query-Writing Exercises

Unique, non-repeating questions covering SQL fundamentals, MySQL architecture, joins, indexing, normalization, transactions, views, stored procedures, triggers, performance tuning, and real-world

troubleshooting & scenario-based query writing.

Contents

1. SQL Fundamentals *(Q1–Q10)*
2. Data Types *(Q11–Q20)*
3. DDL, DML, DCL and TCL Commands *(Q21–Q30)*
4. Joins *(Q31–Q40)*
5. Subqueries and CTEs *(Q41–Q50)*
6. Keys and Constraints *(Q51–Q60)*
7. Database Normalization *(Q61–Q70)*
8. Indexes *(Q71–Q80)*
9. Transactions and ACID Properties *(Q81–Q90)*
10. Views *(Q91–Q100)*
11. Stored Procedures and Functions *(Q101–Q110)*
12. Triggers *(Q111–Q120)*
13. Aggregate, String, and Date Functions *(Q121–Q130)*
14. Performance Tuning and Optimization *(Q131–Q140)*
15. MySQL Architecture and Advanced Topics *(Q141–Q150)*
16. Scenario-Based Questions & Query Writing *(Q151–Q175)*
17. Quick-Fire Concept Refreshers *(Q176–Q177)*
18. Indexing Scenarios *(Q178–Q180)*
19. Views & Data Security Scenarios *(Q181–Q182)*
20. Advanced Troubleshooting Scenarios *(Q183–Q187)*

1. SQL Fundamentals

Q1. What is SQL?

Answer: SQL (Structured Query Language) is a standard language used to create, read, update, and delete data in a relational database. It also lets you define schema, manage access, and control transactions.

Q2. What is a database?

Answer: A database is an organized collection of structured data stored electronically, typically managed by a Database Management System (DBMS) that allows storage, retrieval, and manipulation of that data.

Q3. What is a Relational Database Management System (RDBMS)?

Answer: An RDBMS is a DBMS that stores data in tables made of rows and columns, and enforces relationships between tables using keys. MySQL, PostgreSQL, and Oracle are examples of RDBMS.

Q4. What are the main categories of SQL commands?

Answer: DDL (Data Definition Language: CREATE, ALTER, DROP), DML (Data Manipulation Language: SELECT, INSERT, UPDATE, DELETE), DCL (Data Control Language: GRANT, REVOKE), and TCL (Transaction Control Language: COMMIT, ROLLBACK, SAVEPOINT).

Q5. What is the difference between SQL and MySQL?

Answer: SQL is a query language used to interact with relational databases, while MySQL is an actual RDBMS software product that implements and executes SQL statements to manage data.

Q6. What is a table in SQL?

Answer: A table is a database object that stores data in a structured format of rows (records) and columns (fields), each column having a defined data type.

Q7. What is a schema in MySQL?

Answer: In MySQL, a schema is synonymous with a database — it is a logical container that holds tables, views, procedures, and other database objects.

Q8. What is the difference between CHAR and VARCHAR data types?

Answer: CHAR is a fixed-length string type that always uses the declared number of characters, padding with spaces if needed, while VARCHAR is a variable-length type that stores only the actual characters plus a small length prefix, saving space for variable-sized data.

Q9. What is the difference between DELETE, TRUNCATE, and DROP?

Answer: DELETE removes specific rows using a WHERE clause and can be rolled back; it is logged row by row. TRUNCATE removes all rows quickly by deallocating data pages and resets auto-increment, but cannot use a WHERE clause. DROP removes the entire table structure along with its data permanently.

Q10. What is the difference between WHERE and HAVING clauses?

Answer: WHERE filters individual rows before any grouping occurs and cannot use aggregate functions, whereas HAVING filters groups after the GROUP BY operation and is typically used with aggregate functions like SUM or COUNT.

2. Data Types

Q11. What are the main categories of MySQL data types?

Answer: MySQL data types fall into three broad categories: numeric (INT, DECIMAL, FLOAT), date and time (DATE, DATETIME, TIMESTAMP), and string (CHAR, VARCHAR, TEXT, BLOB).

Q12. What is the difference between FLOAT, DOUBLE, and DECIMAL?

Answer: FLOAT and DOUBLE are approximate floating-point types that can introduce rounding errors, with DOUBLE offering greater precision than FLOAT. DECIMAL is an exact fixed-point type ideal for financial calculations where precision matters.

Q13. What is the difference between DATETIME and TIMESTAMP?

Answer: DATETIME stores a date and time value independent of time zone with a range from 1000 to 9999, while TIMESTAMP stores values relative to UTC, is time-zone aware, and has a smaller range (1970 to 2038), making it useful for tracking record creation or modification times.

Q14. What is the difference between TEXT and BLOB?

Answer: TEXT stores large amounts of character (string) data and is compared using the column's character set, while BLOB stores binary data such as images or files and is compared byte by byte without character set conversion.

Q15. What is the ENUM data type in MySQL?

Answer: ENUM is a string object whose value is chosen from a predefined list of allowed values specified at table creation, useful for storing a fixed set of categorical values like 'small', 'medium', 'large' in a compact form.

Q16. What is the difference between signed and unsigned integer types?

Answer: A signed integer can store both negative and positive values within its range, while an unsigned integer only stores non-negative values, effectively doubling the maximum positive value it can represent for the same storage size.

Q17. What is the NULL value in SQL?

Answer: NULL represents the absence of a value or an unknown value in a column. It is not equal to zero or an empty string, and comparisons with NULL using standard operators return unknown, which is why IS NULL or IS NOT NULL must be used to test for it.

Q18. How do you store JSON data in MySQL?

Answer: MySQL provides a native JSON data type (since version 5.7) that validates the JSON document on insertion and stores it in an optimized binary format, allowing efficient querying with functions like JSON_EXTRACT and the -> operator.

Q19. What is the AUTO_INCREMENT attribute used for?

Answer: AUTO_INCREMENT automatically generates a unique, sequentially increasing numeric value for a column, typically the primary key, each time a new row is inserted without explicitly specifying that column's value.

Q20. What is the difference between DATE, TIME, and YEAR data types?

Answer: DATE stores only a calendar date (YYYY-MM-DD), TIME stores only a time of day or duration (HHH:MM:SS), and YEAR stores a year value in a compact 1-byte format, used when only the year is relevant.

3. DDL, DML, DCL and TCL Commands

Q21. What is the syntax and purpose of the CREATE TABLE statement?

Answer: CREATE TABLE defines a new table's structure, specifying column names, data types, and constraints, for example: CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(50));. It allocates the storage structure without inserting data.

Q22. What is the difference between ALTER and UPDATE?

Answer: ALTER is a DDL command used to change a table's structure, such as adding or dropping columns, while UPDATE is a DML command used to modify the actual data values within existing rows.

Q23. How do you add and drop a column in an existing table?

Answer: You add a column with ALTER TABLE table_name ADD COLUMN column_name datatype; and remove one with ALTER TABLE table_name DROP COLUMN column_name;.

Q24. What is the difference between INSERT INTO ... VALUES and INSERT INTO ... SELECT?

Answer: INSERT INTO ... VALUES inserts one or more explicitly specified rows of literal data, while INSERT INTO ... SELECT inserts rows produced by a SELECT query, allowing data to be copied from another table.

Q25. What is the purpose of the GRANT and REVOKE statements?

Answer: GRANT assigns specific privileges (such as SELECT, INSERT, or ALL) to a database user or role, while REVOKE removes previously granted privileges, together forming the DCL used to manage database access control.

Q26. What are COMMIT and ROLLBACK used for?

Answer: COMMIT permanently saves all changes made during the current transaction to the database, while ROLLBACK undoes all changes made since the last COMMIT, restoring the database to its previous consistent state.

Q27. What is a SAVEPOINT in a transaction?

Answer: A SAVEPOINT marks an intermediate point within a transaction to which you can later roll back using ROLLBACK TO SAVEPOINT, without undoing the entire transaction.

Q28. What is the difference between RENAME TABLE and ALTER TABLE ... RENAME?

Answer: RENAME TABLE can rename one or more tables in a single atomic statement and even move tables between databases, whereas ALTER TABLE ... RENAME is used to rename a single table as part of a broader ALTER TABLE operation.

Q29. How do you modify a column's data type in MySQL?

Answer: You use ALTER TABLE table_name MODIFY COLUMN column_name new_datatype; to change a column's data type or attributes while keeping its name, or CHANGE COLUMN if you also want to rename it.

Q30. What is the difference between a temporary table and a regular table?

Answer: A temporary table, created with CREATE TEMPORARY TABLE, is visible only within the session that created it and is automatically dropped when that session ends, whereas a regular table persists permanently and is accessible to all authorized users.

4. Joins

Q31. What is a JOIN in SQL?

Answer: A JOIN combines rows from two or more tables based on a related column between them, allowing data spread across multiple tables to be retrieved together in a single result set.

Q32. What is the difference between INNER JOIN and OUTER JOIN?

Answer: INNER JOIN returns only rows that have matching values in both tables, while OUTER JOIN (LEFT, RIGHT, or FULL) returns matching rows plus unmatched rows from one or both tables, filling in NULLs where no match exists.

Q33. What is a LEFT JOIN?

Answer: A LEFT JOIN returns all rows from the left table and the matched rows from the right table; if there is no match, NULL values are returned for columns from the right table.

Q34. What is a RIGHT JOIN?

Answer: A RIGHT JOIN returns all rows from the right table and the matched rows from the left table; unmatched rows from the left table are filled with NULL.

Q35. Does MySQL support FULL OUTER JOIN?

Answer: MySQL does not natively support FULL OUTER JOIN, but the same result can be achieved by combining a LEFT JOIN and a RIGHT JOIN using UNION.

Q36. What is a SELF JOIN?

Answer: A SELF JOIN is a join of a table with itself, typically using table aliases to distinguish the two instances, useful for comparing rows within the same table, such as finding employees and their managers in one table.

Q37. What is a CROSS JOIN?

Answer: A CROSS JOIN produces the Cartesian product of two tables, pairing every row of the first table with every row of the second table, resulting in rows equal to the product of both table sizes.

Q38. What is the difference between JOIN and UNION?

Answer: JOIN combines columns from multiple tables side by side based on a related key, while UNION combines rows from multiple SELECT queries vertically, stacking their result sets and requiring compatible column counts and types.

Q39. What happens if you JOIN two tables without specifying a join condition?

Answer: Without a join condition, the JOIN behaves like a CROSS JOIN, producing a Cartesian product where every row of the first table is paired with every row of the second table.

Q40. Can you join more than two tables in a single query?

Answer: Yes, you can chain multiple JOIN clauses in a single query to combine data from three or more tables, as long as each join specifies its own condition linking the tables.

5. Subqueries and CTEs

Q41. What is a subquery?

Answer: A subquery is a query nested inside another SQL statement (SELECT, INSERT, UPDATE, or DELETE), enclosed in parentheses, used to return data that the outer query then uses for filtering or computation.

Q42. What is the difference between a correlated and a non-correlated subquery?

Answer: A non-correlated subquery executes independently of the outer query and runs only once, while a correlated subquery references columns from the outer query and is re-evaluated once for every row processed by the outer query.

Q43. What is the difference between a subquery and a JOIN?

Answer: A JOIN combines columns from related tables into a single flat result set and is often more efficient for retrieving combined data, while a subquery is a nested, self-contained query typically used for filtering, existence checks, or computing a single value, and can sometimes be less optimizable by the query planner.

Q44. What is a Common Table Expression (CTE)?

Answer: A CTE, defined using the WITH clause, is a named temporary result set that exists only for the duration of a single query, improving readability and allowing complex queries to be broken into logical, reusable steps.

Q45. What is a recursive CTE and when would you use one?

Answer: A recursive CTE references itself to repeatedly process hierarchical or tree-structured data, such as generating organizational charts or traversing a category tree, until a termination condition stops the recursion.

Q46. What is the EXISTS operator used for?

Answer: EXISTS tests whether a subquery returns any rows at all, returning TRUE if at least one row is found; it is often more efficient than IN for large datasets because it can stop scanning as soon as one match is found.

Q47. What is the difference between IN and EXISTS?

Answer: IN compares a value against a list or subquery result and can behave unpredictably with NULLs, while EXISTS simply checks for the presence of any matching row and generally performs better with correlated subqueries on large tables.

Q48. Can a subquery return multiple columns?

Answer: Yes, a subquery can return multiple columns when used in contexts like row constructors for comparison, for example WHERE (col1, col2) IN (SELECT a, b FROM table2), though scalar subqueries used in expressions must return exactly one column and one row.

Q49. What is a scalar subquery?

Answer: A scalar subquery is a subquery that returns exactly a single row and a single column, so its result can be used anywhere a single value expression is expected, such as in a SELECT list or a comparison.

Q50. How do derived tables differ from CTEs?

Answer: A derived table is a subquery used in the FROM clause and given an alias, existing only for that query, while a CTE serves a similar purpose but is defined upfront with WITH, can be referenced multiple times in the same query, and supports recursion.

6. Keys and Constraints

Q51. What is a Primary Key?

Answer: A Primary Key is a column or set of columns that uniquely identifies each row in a table; it cannot contain NULL values and each table can have only one primary key, which is automatically indexed.

Q52. What is a Foreign Key?

Answer: A Foreign Key is a column that establishes a link between data in two tables by referencing the Primary Key of another table, enforcing referential integrity so that a value in the foreign key column must exist in the referenced table.

Q53. What is a Unique Key/Constraint?

Answer: A Unique constraint ensures that all values in a column (or combination of columns) are distinct across the table, similar to a Primary Key but it allows one NULL value and a table can have multiple unique constraints.

Q54. What is a Composite Key?

Answer: A Composite Key is a primary key made up of two or more columns combined, used when a single column is not sufficient to uniquely identify a row.

Q55. What is the difference between a Primary Key and a Unique Key?

Answer: A Primary Key does not allow NULL values and only one exists per table, while a Unique Key allows a single NULL value (per column) and a table can have multiple unique keys.

Q56. What is a CHECK constraint?

Answer: A CHECK constraint enforces a condition that every row must satisfy for a specific column, such as CHECK (age >= 18), rejecting any INSERT or UPDATE that would violate the condition.

Q57. What is a Candidate Key?

Answer: A Candidate Key is any column or combination of columns that could qualify as the primary key because it uniquely identifies rows; one candidate key is selected as the primary key and the rest become alternate keys.

Q58. What is referential integrity?

Answer: Referential integrity is a database property ensuring that relationships between tables remain consistent, meaning a foreign key value must always correspond to an existing primary key value in the referenced table, or be NULL.

Q59. What are ON DELETE CASCADE and ON UPDATE CASCADE?

Answer: These are foreign key actions that automatically propagate changes: ON DELETE CASCADE deletes child rows when the referenced parent row is deleted, and ON UPDATE CASCADE updates child foreign key values when the parent's key value changes.

Q60. What is a NOT NULL constraint?

Answer: A NOT NULL constraint ensures that a column cannot store a NULL value, forcing every row to have a defined value for that field.

7. Database Normalization

Q61. What is normalization?

Answer: Normalization is the process of organizing a database's tables and columns to minimize data redundancy and avoid update, insertion, and deletion anomalies, typically by decomposing large tables into smaller, related ones.

Q62. What is First Normal Form (1NF)?

Answer: 1NF requires that each column hold atomic (indivisible) values, each row be unique, and there be no repeating groups or arrays within a single column.

Q63. What is Second Normal Form (2NF)?

Answer: 2NF requires that a table be in 1NF and that every non-key attribute be fully functionally dependent on the entire primary key, eliminating partial dependencies that occur with composite keys.

Q64. What is Third Normal Form (3NF)?

Answer: 3NF requires that a table be in 2NF and that no non-key attribute depends on another non-key attribute, eliminating transitive dependencies so every non-key column depends only on the primary key.

Q65. What is Boyce-Codd Normal Form (BCNF)?

Answer: BCNF is a stricter version of 3NF that requires every determinant in the table to be a candidate key, resolving certain anomalies that can still exist in 3NF when there are overlapping candidate keys.

Q66. What is denormalization and why is it used?

Answer: Denormalization is the deliberate process of combining tables or introducing redundancy into an already normalized database to improve read performance by reducing the number of joins needed, at the cost of some data redundancy.

Q67. What is a functional dependency?

Answer: A functional dependency exists between two attributes when the value of one attribute (the determinant) uniquely determines the value of another attribute in the same table.

Q68. What is a transitive dependency?

Answer: A transitive dependency occurs when a non-key column depends on another non-key column rather than depending directly on the primary key, which 3NF is designed to eliminate.

Q69. What are the trade-offs between normalization and denormalization?

Answer: Normalization reduces redundancy and improves data integrity but can require more joins and slower reads, while denormalization improves read performance by reducing joins but increases storage use and the risk of data inconsistency during updates.

Q70. What is the difference between a partial dependency and a full dependency?

Answer: A partial dependency occurs when a non-key attribute depends on only part of a composite primary key, whereas a full functional dependency means the attribute depends on the entire composite key, which 2NF requires.

8. Indexes

Q71. What is an index in MySQL?

Answer: An index is a separate data structure, typically a B-Tree, built on one or more columns of a table that speeds up data retrieval by allowing the database to locate rows without scanning the entire table.

Q72. What are the different types of indexes in MySQL?

Answer: MySQL supports Primary key indexes, Unique indexes, regular (non-unique) indexes, Composite (multi-column) indexes, Full-text indexes for text search, and Spatial indexes for geographic data.

Q73. What is a clustered index versus a non-clustered index?

Answer: A clustered index determines the physical storage order of table rows and there can be only one per table (in InnoDB it is the primary key), while a non-clustered (secondary) index is a separate structure that stores pointers back to the actual row data.

Q74. What is a composite (multi-column) index?

Answer: A composite index is built on two or more columns together, and it is most effective when queries filter or sort using a leftmost prefix of those columns in the same order they were defined.

Q75. What are the advantages and disadvantages of using indexes?

Answer: Indexes speed up SELECT queries and WHERE/JOIN/ORDER BY operations, but they consume additional disk space and slow down INSERT, UPDATE, and DELETE operations because the index structures must also be updated.

Q76. What is a covering index?

Answer: A covering index is an index that contains all the columns needed to satisfy a query, allowing MySQL to retrieve the result entirely from the index without accessing the actual table data, improving performance.

Q77. How do you view the indexes on a table in MySQL?

Answer: You can run SHOW INDEX FROM table_name; or query the INFORMATION_SCHEMA.STATISTICS table to see the indexes defined on a given table.

Q78. What is the EXPLAIN statement used for?

Answer: EXPLAIN shows the execution plan MySQL will use for a query, revealing which indexes are used, the join order, estimated rows scanned, and other details that help identify performance bottlenecks.

Q79. When should you avoid creating an index on a column?

Answer: Avoid indexing columns with low cardinality (few distinct values like a boolean flag), columns that change frequently, very small tables, or columns rarely used in WHERE, JOIN, or ORDER BY clauses, since the maintenance overhead may outweigh the benefit.

Q80. What is a full-text index?

Answer: A full-text index is a special type of index used for efficient searching of natural language text within CHAR, VARCHAR, or TEXT columns, enabling queries with the MATCH() AGAINST() syntax instead of slow LIKE '%word%' scans.

9. Transactions and ACID Properties

Q81. What is a transaction in a database?

Answer: A transaction is a logical unit of work consisting of one or more SQL statements that are executed as a single, indivisible operation, ensuring all statements succeed together or none of them take effect.

Q82. What does ACID stand for?

Answer: ACID stands for Atomicity (all-or-nothing execution), Consistency (transactions bring the database from one valid state to another), Isolation (concurrent transactions don't interfere with each other), and Durability (committed changes survive system failures).

Q83. What is the difference between COMMIT and ROLLBACK?

Answer: COMMIT permanently saves all changes made during the current transaction, while ROLLBACK discards all changes made since the transaction began or since the last savepoint.

Q84. What are the isolation levels available in MySQL?

Answer: MySQL (InnoDB) supports four standard isolation levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ (the default), and SERIALIZABLE, each offering a different trade-off between concurrency and consistency.

Q85. What is a dirty read?

Answer: A dirty read occurs when a transaction reads data that has been modified by another transaction but not yet committed; if that other transaction rolls back, the first transaction has read invalid data.

Q86. What is a non-repeatable read?

Answer: A non-repeatable read occurs when a transaction reads the same row twice and gets different values because another transaction modified and committed changes to that row in between the two reads.

Q87. What is a phantom read?

Answer: A phantom read occurs when a transaction re-executes a query returning a set of rows and finds that the set has changed because another transaction inserted or deleted rows matching the query's condition in the meantime.

Q88. What storage engine does MySQL use for transactions and why?

Answer: InnoDB is MySQL's default transactional storage engine; it supports ACID-compliant transactions, row-level locking, and foreign key constraints, unlike the older MyISAM engine which does not support transactions.

Q89. What is deadlock in MySQL and how can it be avoided?

Answer: A deadlock occurs when two or more transactions each hold a lock the other needs, causing them to wait indefinitely; MySQL detects deadlocks and automatically rolls back one transaction. It can be minimized by accessing tables/rows in a consistent order, keeping transactions short, and using appropriate indexes.

Q90. What is the difference between autocommit mode being ON and OFF?

Answer: With autocommit ON (MySQL's default), every individual SQL statement is automatically committed as its own transaction; with autocommit OFF, changes accumulate until an explicit COMMIT or ROLLBACK is issued, allowing multiple statements to be grouped into one transaction.

10. Views

Q91. What is a View in SQL?

Answer: A View is a virtual table based on the result of a stored SELECT query; it does not store data itself (in most cases) but presents data from one or more underlying tables in a simplified or restricted form.

Q92. What are the advantages of using views?

Answer: Views simplify complex queries by hiding underlying joins and logic, provide a layer of security by restricting access to specific columns or rows, and offer a consistent, reusable interface even if the underlying table structure changes.

Q93. Can you update data through a view?

Answer: Yes, but only if the view is 'updatable,' meaning it is based on a single table (or a simple join in some cases) without aggregate functions, GROUP BY, DISTINCT, or UNION; updates through the view then modify the underlying base table.

Q94. What is the difference between a view and a table?

Answer: A table physically stores data on disk, while a view is a saved query definition that dynamically retrieves data from underlying tables each time it is queried, without storing its own separate data set.

Q95. How do you create and drop a view in MySQL?

Answer: You create a view with CREATE VIEW view_name AS SELECT ...; and remove it with DROP VIEW view_name;.

Q96. What is a materialized view, and does MySQL support it?

Answer: A materialized view physically stores the query result and must be manually or periodically refreshed, improving read performance at the cost of data freshness. MySQL does not natively support materialized views, but similar behavior can be simulated using a regular table populated via scheduled events or triggers.

Q97. Can a view be based on multiple tables?

Answer: Yes, a view can be defined using a SELECT statement that joins multiple tables, though such views are often read-only if the join is complex.

Q98. What is the WITH CHECK OPTION clause used for in views?

Answer: WITH CHECK OPTION ensures that any INSERT or UPDATE made through the view must satisfy the view's WHERE clause condition, preventing rows from being modified in a way that would make them disappear from the view's result set.

Q99. Do views improve query performance?

Answer: Views themselves do not inherently improve performance since MySQL re-executes the underlying query each time (unless it is a materialized simulation); their main benefits are simplification, security, and abstraction rather than speed.

Q100. Can you create an index on a view in MySQL?

Answer: No, MySQL does not support creating indexes directly on views since they are not physically stored; performance instead depends on the indexes present on the view's underlying base tables.

11. Stored Procedures and Functions

Q101. What is a stored procedure?

Answer: A stored procedure is a precompiled, named collection of SQL statements stored in the database that can be executed repeatedly with the CALL statement, optionally accepting input and output parameters.

Q102. What is the difference between a stored procedure and a function in MySQL?

Answer: A stored procedure can perform actions like INSERT/UPDATE, may or may not return a value, and is invoked with CALL, while a function must return exactly one value, can be used directly within SQL expressions like SELECT statements, and cannot modify data with statements that aren't allowed inside functions.

Q103. What are the advantages of using stored procedures?

Answer: Stored procedures improve performance by reducing network traffic (compiled logic runs on the server), promote code reuse and consistency, enhance security by restricting direct table access, and simplify maintenance since logic is centralized.

Q104. How do you create a simple stored procedure in MySQL?

Answer: Using the DELIMITER command to change the statement terminator, then CREATE PROCEDURE proc_name(IN param INT) BEGIN ... END;, for example: DELIMITER $$ CREATE PROCEDURE GetEmployee(IN empId INT) BEGIN SELECT * FROM employees WHERE id = empId; END$$ DELIMITER ;.

Q105. What are IN, OUT, and INOUT parameters in stored procedures?

Answer: IN parameters pass a value into the procedure (default), OUT parameters return a value from the procedure back to the caller, and INOUT parameters both pass a value in and allow the procedure to return a modified value through the same parameter.

Q106. What is a user-defined function (UDF) in MySQL?

Answer: A user-defined function is a custom function created with CREATE FUNCTION that encapsulates reusable logic, accepts parameters, and returns a single value, which can then be used inside SQL statements just like a built-in function.

Q107. What is the DELIMITER command used for in MySQL scripts?

Answer: DELIMITER temporarily changes the statement terminator (usually from ; to something like $$) so that the semicolons used inside a stored procedure or trigger body are not misinterpreted as the end of the CREATE statement itself.

Q108. Can a stored procedure call another stored procedure?

Answer: Yes, stored procedures can call other stored procedures, allowing complex logic to be broken into smaller, modular, reusable units.

Q109. How do you handle errors within a MySQL stored procedure?

Answer: MySQL provides DECLARE ... HANDLER statements to catch specific conditions such as SQLEXCEPTION or SQLWARNING, allowing you to define custom error-handling logic like logging the error or rolling back a transaction when an issue occurs.

Q110. What are the drawbacks of using stored procedures?

Answer: Stored procedures can be harder to version control and debug than application code, may lead to vendor lock-in since syntax differs across database systems, and can make business logic harder to test outside the database.

12. Triggers

Q111. What is a trigger in MySQL?

Answer: A trigger is a database object that automatically executes a predefined set of SQL statements in response to a specific event (INSERT, UPDATE, or DELETE) occurring on a specified table.

Q112. What are the types of triggers based on timing?

Answer: Triggers can be BEFORE triggers, which execute prior to the triggering event (often used for validation), or AFTER triggers, which execute after the event has completed (often used for logging or cascading actions).

Q113. What is the difference between OLD and NEW in a trigger?

Answer: Inside a trigger, OLD refers to the row's values before the change (available in UPDATE and DELETE triggers), while NEW refers to the row's values after the change (available in INSERT and UPDATE triggers).

Q114. Can a table have multiple triggers for the same event?

Answer: Yes, MySQL allows multiple triggers of the same type (e.g., multiple BEFORE INSERT triggers) on one table, and you can control their execution order using the FOLLOWS or PRECEDES clause.

Q115. What are common use cases for triggers?

Answer: Triggers are commonly used for auditing/logging changes to a table, enforcing complex business rules that constraints cannot express, automatically updating related tables, and maintaining derived or summary data.

Q116. What are the disadvantages of using triggers?

Answer: Triggers can make debugging difficult since their execution is implicit and not visible in application code, can introduce hidden performance overhead on every DML operation, and can complicate the logic flow when triggers cascade into other triggers.

Q117. Can a trigger call a stored procedure?

Answer: Yes, a trigger's body can call a stored procedure to execute more complex or reusable logic in response to the triggering event.

Q118. How do you view existing triggers in a MySQL database?

Answer: You can run SHOW TRIGGERS; to list triggers in the current database, or query the INFORMATION_SCHEMA.TRIGGERS table for more detailed metadata.

Q119. How do you drop a trigger in MySQL?

Answer: You remove a trigger using the statement DROP TRIGGER [IF EXISTS] trigger_name;.

Q120. Can triggers be used to enforce referential integrity instead of foreign keys?

Answer: While technically possible, it is not recommended; foreign key constraints are more efficient, declarative, and reliably enforced by the storage engine, whereas triggers add procedural overhead and are more error-prone for this purpose.

13. Aggregate, String, and Date Functions

Q121. What are aggregate functions in SQL? Give examples.

Answer: Aggregate functions perform a calculation across a set of rows and return a single value. Common examples in MySQL include COUNT(), SUM(), AVG(), MIN(), and MAX().

Q122. What is the difference between COUNT(*), COUNT(column), and COUNT(DISTINCT column)?

Answer: COUNT(*) counts all rows regardless of NULLs, COUNT(column) counts only rows where that specific column is not NULL, and COUNT(DISTINCT column) counts the number of unique non-NULL values in that column.

Q123. What is the GROUP BY clause used for?

Answer: GROUP BY groups rows that share the same values in specified columns into summary rows, typically used alongside aggregate functions to compute statistics like totals or averages per group.

Q124. What is the difference between GROUP BY and ORDER BY?

Answer: GROUP BY groups rows together to allow aggregate calculations per group, while ORDER BY simply sorts the final result set in ascending or descending order without changing how rows are grouped or aggregated.

Q125. What are some commonly used MySQL string functions?

Answer: Common string functions include CONCAT() to join strings, SUBSTRING() to extract part of a string, LENGTH() for string length, UPPER()/LOWER() for case conversion, TRIM() to remove whitespace, and REPLACE() to substitute text.

Q126. What are some commonly used MySQL date functions?

Answer: Common date functions include NOW() and CURDATE() for the current date/time, DATEDIFF() to calculate the difference between two dates, DATE_ADD()/DATE_SUB() to add or subtract intervals, and DATE_FORMAT() to format a date as a string.

Q127. What is the difference between NOW() and CURDATE() in MySQL?

Answer: NOW() returns the current date and time together (YYYY-MM-DD HH:MM:SS), while CURDATE() returns only the current date without any time component.

Q128. What does the COALESCE() function do?

Answer: COALESCE() returns the first non-NULL value from a list of expressions passed to it, commonly used to substitute a default value when a column might contain NULL.

Q129. What is the difference between IFNULL() and COALESCE() in MySQL?

Answer: IFNULL() is a MySQL-specific function that accepts exactly two arguments and returns the second if the first is NULL, while COALESCE() is the standard SQL function that accepts any number of arguments and returns the first non-NULL one, making it more flexible and portable.

Q130. What is a window function and how does it differ from an aggregate function?

Answer: A window function, used with the OVER() clause, performs a calculation across a set of related rows (a 'window') like an aggregate does, but unlike GROUP BY it does not collapse the rows into a single output row — each row retains its individual identity alongside the computed value, e.g., RANK(), ROW_NUMBER(), and SUM() OVER(PARTITION BY ...).

14. Performance Tuning and Optimization

Q131. How would you optimize a slow-running SQL query?

Answer: Start by running EXPLAIN to review the execution plan, ensure appropriate indexes exist on columns used in WHERE/JOIN/ORDER BY, avoid SELECT *, rewrite inefficient subqueries as joins where possible, and check for full table scans or missing statistics.

Q132. What is the MySQL slow query log?

Answer: The slow query log is a MySQL feature that records queries taking longer than a configured threshold (long_query_time) to execute, helping developers identify and diagnose performance bottlenecks.

Q133. What is query caching and is it still recommended in modern MySQL?

Answer: Query caching stored the result of a SELECT query so an identical subsequent query could be served instantly without re-execution; however, MySQL's built-in query cache was deprecated in MySQL 5.7 and removed in MySQL 8.0 due to scalability issues under concurrent writes, so application-level or external caching (like Redis) is now recommended instead.

Q134. What is the difference between a full table scan and an index scan?

Answer: A full table scan reads every row in a table to find matches, which is slow for large tables, while an index scan uses an index structure to jump directly to relevant rows, which is generally much faster for selective queries.

Q135. How does the LIMIT clause help with performance?

Answer: LIMIT restricts the number of rows returned by a query, reducing the amount of data MySQL needs to process and transfer, which is especially useful for pagination and previewing large result sets.

Q136. What is database connection pooling and why does it matter?

Answer: Connection pooling maintains a reusable pool of open database connections that application threads borrow and return, avoiding the overhead of establishing a new TCP/authentication handshake for every query, which significantly improves throughput under load.

Q137. What is the N+1 query problem and how do you avoid it?

Answer: The N+1 problem occurs when an application runs one query to fetch a list of records and then executes a separate query for each record to fetch related data, resulting in N+1 total queries; it can be avoided by using JOINs or batching related lookups into a single query.

Q138. What is partitioning in MySQL?

Answer: Partitioning splits a large table's data into smaller, more manageable physical pieces (partitions) based on a defined rule such as a range or list of values, while the table still behaves as a single logical unit, which can improve query performance and maintenance for very large tables.

Q139. What is the difference between vertical scaling and horizontal scaling for databases?

Answer: Vertical scaling increases a single server's capacity by adding more CPU, RAM, or storage, whereas horizontal scaling adds more database servers (e.g., through sharding or read replicas) to distribute load across multiple machines.

Q140. What is database replication and what are its benefits?

Answer: Replication copies data from a primary (source) database server to one or more replica servers, providing benefits such as high availability through failover, load distribution by directing read queries to replicas, and offsite backups for disaster recovery.

15. MySQL Architecture and Advanced Topics

Q141. What are the different storage engines available in MySQL?

Answer: MySQL supports multiple pluggable storage engines, the most common being InnoDB (default, transactional, supports foreign keys and row-level locking) and MyISAM (non-transactional, table-level locking, historically faster for read-heavy workloads but lacking crash recovery and referential integrity).

Q142. What is the difference between InnoDB and MyISAM?

Answer: InnoDB supports transactions, foreign keys, and row-level locking with crash recovery, making it suitable for write-heavy or critical applications, while MyISAM offers table-level locking, no transaction support, and no foreign key enforcement, but was historically favored for simple, read-heavy workloads.

Q143. What is database sharding?

Answer: Sharding is a horizontal partitioning technique that splits a large database into smaller, independent pieces called shards, each hosted on a separate server, distributing data and load across multiple machines to improve scalability.

Q144. What is the difference between horizontal and vertical partitioning?

Answer: Horizontal partitioning (sharding) splits a table's rows across multiple tables or servers, while vertical partitioning splits a table's columns into separate tables, often grouping frequently accessed columns together for performance.

Q145. What is master-slave (primary-replica) replication in MySQL?

Answer: In master-slave replication, all write operations occur on the primary/master server, and its changes are asynchronously (or semi-synchronously) copied to one or more replica/slave servers, which can then be used to serve read queries or serve as failover backups.

Q146. What is the binary log (binlog) in MySQL used for?

Answer: The binary log records all statements or row-level changes that modify data, and it is used for replication (replicas read the binlog to apply changes) as well as for point-in-time data recovery.

Q147. What is a database lock, and what are the common types?

Answer: A lock is a mechanism that controls concurrent access to data to maintain consistency. Common types include shared (read) locks, which allow multiple transactions to read simultaneously, and exclusive (write) locks, which prevent other transactions from reading or writing the locked resource, applied at either row or table level.

Q148. What is the difference between optimistic and pessimistic locking?

Answer: Pessimistic locking locks a resource as soon as a transaction accesses it, preventing others from modifying it until the lock is released, while optimistic locking allows concurrent access and only checks for conflicts (often via a version column) at commit time, rolling back if another transaction modified the data first.

Q149. What is the CAP theorem and how does it relate to relational databases?

Answer: The CAP theorem states that a distributed data system can only guarantee two of three properties at once — Consistency, Availability, and Partition tolerance. Traditional single-node relational

databases like MySQL prioritize consistency, while distributed/NoSQL systems often make different trade-offs depending on their design goals.

Q150. What is the difference between SQL and NoSQL databases?

Answer: SQL databases are relational, use structured schemas with tables and fixed columns, and enforce ACID properties, making them well-suited for structured data with complex relationships, while NoSQL databases (document, key-value, column, or graph stores) offer flexible or schema-less data models and are often designed to prioritize horizontal scalability and availability over strict consistency.

16. Scenario-Based Questions & Query Writing

The following questions present real-world scenarios and require writing the actual SQL query to solve them — the most common format used in practical MySQL/SQL interviews.

Q151. You have an Employees table (id, name, salary, dept_id). Write a query to find the second highest salary.

Setup: *Table: Employees(id, name, salary, dept_id)*

Query:

```
SELECT MAX(salary) AS second_highest_salary
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

Explanation: This finds the maximum salary that is strictly less than the overall maximum, which is the second highest value. Using LIMIT/OFFSET (ORDER BY salary DESC LIMIT 1 OFFSET 1) is an alternative, but it can return the wrong result if there are duplicate top salaries, whereas this approach handles ties correctly.

Q152. Write a query to find the Nth highest salary in a table, where N is a parameter.

Setup: *Table: Employees(id, name, salary)*

Query:

```
SELECT DISTINCT salary
FROM Employees e1
WHERE (N - 1) = (
  SELECT COUNT(DISTINCT salary)
  FROM Employees e2
  WHERE e2.salary > e1.salary
);

-- Alternative using DENSE_RANK() (MySQL 8.0+):
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employees
) ranked
WHERE rnk = N;
```

Explanation: The correlated subquery counts how many distinct salaries are greater than the current row's salary; when that count equals N-1, the current salary is the Nth highest. In MySQL 8.0+, DENSE_RANK() is a cleaner and more efficient way to solve the same 'Nth highest' class of problems while correctly handling duplicate salaries.

Q153. How would you find and remove duplicate rows from a table, keeping only one copy of each?

Setup: *Table: Customers(id, email, name) -- 'email' should be unique but has duplicates*

Query:

```
-- Step 1: Identify duplicates
SELECT email, COUNT(*) AS cnt
FROM Customers
GROUP BY email
HAVING COUNT(*) > 1;
```

```
-- Step 2: Delete duplicates, keeping the lowest id per email
DELETE c1 FROM Customers c1
INNER JOIN Customers c2
  ON c1.email = c2.email AND c1.id > c2.id;
```

Explanation: First, GROUP BY combined with HAVING COUNT(*) > 1 identifies which values are duplicated. The DELETE then self-joins the table to itself on the duplicate key and removes any row whose id is greater than another row sharing the same email, effectively keeping only the earliest (lowest id) record for each email.

Q154. Write a query to find employees who earn more than their direct manager.

Setup: *Table: Employees(id, name, salary, manager_id) -- manager_id references Employees.id*

Query:

```
SELECT e.name AS employee, e.salary AS employee_salary,
       m.name AS manager, m.salary AS manager_salary
FROM Employees e
JOIN Employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Explanation: This is a classic self-join scenario. The Employees table is joined to itself: once as 'e' representing the employee, and once as 'm' representing that employee's manager (matched via manager_id = id). The WHERE clause then filters for cases where the employee's salary exceeds their manager's.

Q155. Write a query to find all customers who have never placed an order.

Setup: *Tables: Customers(id, name), Orders(id, customer_id, order_date)*

Query:

```
SELECT c.id, c.name
FROM Customers c
LEFT JOIN Orders o ON c.id = o.customer_id
WHERE o.id IS NULL;

-- Alternative using NOT EXISTS:
SELECT c.id, c.name
FROM Customers c
WHERE NOT EXISTS (
  SELECT 1 FROM Orders o WHERE o.customer_id = c.id
);
```

Explanation: A LEFT JOIN keeps every customer row regardless of whether a matching order exists, and unmatched rows produce NULL in the Orders columns; filtering WHERE o.id IS NULL isolates customers with no matching order. NOT EXISTS achieves the same result and often performs better on large tables since it can stop searching as soon as one match is found.

Q156. Write a query to find the department with the highest average salary.

Setup: *Tables: Employees(id, name, salary, dept_id), Departments(id, dept_name)*

Query:

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM Employees e
JOIN Departments d ON e.dept_id = d.id
GROUP BY d.dept_name
```

```
ORDER BY avg_salary DESC
LIMIT 1;
```

Explanation: GROUP BY dept_name aggregates employees per department so AVG(salary) can be computed for each group. Sorting the results in descending order of average salary and limiting to one row returns the single highest-paying department on average.

Q157. Write a query to calculate a running total of monthly sales.

Setup: *Table: Sales(id, sale_month, amount)*

Query:

```
SELECT sale_month, amount,
       SUM(amount) OVER (ORDER BY sale_month) AS running_total
FROM Sales
ORDER BY sale_month;
```

Explanation: The window function SUM(amount) OVER (ORDER BY sale_month) computes a cumulative sum: for each row, it adds the current row's amount to the sum of all preceding rows in the ORDER BY sequence, without collapsing rows the way a GROUP BY would.

Q158. Write a query to find the top 3 highest-paid employees in each department.

Setup: *Table: Employees(id, name, salary, dept_id)*

Query:

```
SELECT id, name, salary, dept_id
FROM (
  SELECT id, name, salary, dept_id,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM Employees
) ranked
WHERE rnk <= 3;
```

Explanation: PARTITION BY dept_id restarts the ranking for each department, and ORDER BY salary DESC ranks employees within each partition from highest to lowest salary. DENSE_RANK() assigns the same rank to tied salaries without skipping numbers, and wrapping this in a subquery lets the outer query filter to the top 3 per department.

Q159. Write a query to find users who logged in on three or more consecutive days.

Setup: *Table: Logins(user_id, login_date)*

Query:

```
SELECT DISTINCT l1.user_id
FROM Logins l1
JOIN Logins l2
  ON l1.user_id = l2.user_id AND l2.login_date = DATE_ADD(l1.login_date, INTERVAL 1 DAY)
JOIN Logins l3
  ON l1.user_id = l3.user_id AND l3.login_date = DATE_ADD(l1.login_date, INTERVAL 2 DAY);
```

Explanation: The table is joined to itself twice: once to find a login exactly one day after the current row's date, and again to find a login exactly two days after. If both matches exist for the same user, that user has three consecutive login dates starting at l1.login_date.

Q160. Write a query to find gaps (missing values) in a sequence of IDs.

Setup: *Table: Orders(id) -- ids should be sequential but some are missing*

Query:

```
SELECT (o1.id + 1) AS gap_start
FROM Orders o1
WHERE NOT EXISTS (
  SELECT 1 FROM Orders o2 WHERE o2.id = o1.id + 1
)
AND o1.id < (SELECT MAX(id) FROM Orders)
ORDER BY gap_start;
```

Explanation: For every existing id, the query checks whether id + 1 also exists in the table using NOT EXISTS; if it doesn't (and the current id isn't the last one overall), then id + 1 marks the start of a gap in the sequence.

Q161. Write a query to pivot rows of yearly sales data into columns (one column per year).

Setup: *Table: Sales(product, year, revenue)*

Query:

```
SELECT product,
  SUM(CASE WHEN year = 2023 THEN revenue ELSE 0 END) AS revenue_2023,
  SUM(CASE WHEN year = 2024 THEN revenue ELSE 0 END) AS revenue_2024,
  SUM(CASE WHEN year = 2025 THEN revenue ELSE 0 END) AS revenue_2025
FROM Sales
GROUP BY product;
```

Explanation: MySQL has no built-in PIVOT operator, so conditional aggregation is used instead: a CASE expression checks the year for each row and routes its revenue into the matching year-specific column, while SUM() with GROUP BY product collapses all rows for a product into one row with each year's total in its own column.

Q162. Write a query to find customers whose total order amount is above the overall average order amount across all customers.

Setup: *Tables: Customers(id, name), Orders(id, customer_id, amount)*

Query:

```
SELECT c.id, c.name, SUM(o.amount) AS total_spent
FROM Customers c
JOIN Orders o ON c.id = o.customer_id
GROUP BY c.id, c.name
HAVING SUM(o.amount) > (
  SELECT AVG(customer_total) FROM (
    SELECT SUM(amount) AS customer_total
    FROM Orders
    GROUP BY customer_id
  ) t
);
```

Explanation: The inner derived table first computes each customer's total spend, and the subquery then averages those totals to get the overall average order amount per customer. The outer query groups orders by customer, sums their spending, and HAVING filters to only those customers whose total exceeds that average.

Q163. Write a query to perform an 'upsert' — insert a new product, or update its stock if it already exists.

Setup: *Table: Products(sku VARCHAR PRIMARY KEY, name, stock)*

Query:

```
INSERT INTO Products (sku, name, stock)
VALUES ('SKU123', 'Wireless Mouse', 50)
ON DUPLICATE KEY UPDATE
  stock = stock + VALUES(stock);
```

Explanation: MySQL's INSERT ... ON DUPLICATE KEY UPDATE attempts the insert first; if it would violate a primary key or unique constraint, it instead performs the specified UPDATE on the existing row. Here, VALUES(stock) refers to the value that was attempted in the INSERT, so the existing stock is incremented rather than overwritten.

Q164. Write a query to retrieve the most recent order for each customer.

Setup: *Table: Orders(id, customer_id, order_date, amount)*

Query:

```
SELECT o.*
FROM Orders o
JOIN (
  SELECT customer_id, MAX(order_date) AS latest_date
  FROM Orders
  GROUP BY customer_id
) latest
  ON o.customer_id = latest.customer_id AND o.order_date = latest.latest_date;

-- Alternative using ROW_NUMBER() (MySQL 8.0+):
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS rn
  FROM Orders o
) t
WHERE rn = 1;
```

Explanation: The first approach uses a derived table to find each customer's latest order_date, then joins back to the Orders table to fetch the full row matching that date. The ROW_NUMBER() alternative partitions rows by customer, numbers them from most recent to oldest, and keeps only the row numbered 1 in each partition.

Q165. Write a query to find products that have never been ordered.

Setup: *Tables: Products(id, name), OrderItems(id, product_id, order_id)*

Query:

```
SELECT p.id, p.name
FROM Products p
LEFT JOIN OrderItems oi ON p.id = oi.product_id
WHERE oi.id IS NULL;
```

Explanation: A LEFT JOIN from Products to OrderItems preserves every product row even when there is no matching order item; filtering for oi.id IS NULL isolates the products that had no matches at all, meaning they have never appeared in an order.

Q166. You need to paginate through a very large Orders table efficiently. Why is OFFSET-based pagination problematic, and how would you fix it?

Setup: *Table: Orders(id, customer_id, order_date)*

Query:

```
-- Problematic for large offsets (MySQL must scan and discard 'offset' rows):
SELECT * FROM Orders ORDER BY id LIMIT 20 OFFSET 100000;

-- Efficient keyset (seek) pagination:
SELECT * FROM Orders
WHERE id > 100000        -- last id seen on the previous page
ORDER BY id
LIMIT 20;
```

Explanation: With LIMIT ... OFFSET, MySQL must still scan and discard all rows before the offset, so performance degrades linearly as the offset grows on large tables. Keyset (seek) pagination instead remembers the last row's key from the previous page and uses a WHERE condition to jump directly to the next set of rows, which stays fast regardless of how deep the pagination goes, since it can leverage an index directly.

Q167. Write a query to calculate year-over-year revenue growth percentage.

Setup: *Table: Revenue(year, total_revenue)*

Query:

```
SELECT r1.year,
       r1.total_revenue,
       r2.total_revenue AS prev_year_revenue,
       ROUND(((r1.total_revenue - r2.total_revenue) / r2.total_revenue) * 100, 2) AS
yoy_growth_pct
FROM Revenue r1
JOIN Revenue r2 ON r1.year = r2.year + 1
ORDER BY r1.year;

-- Alternative using LAG() (MySQL 8.0+):
SELECT year, total_revenue,
       ROUND(((total_revenue - LAG(total_revenue) OVER (ORDER BY year))
              / LAG(total_revenue) OVER (ORDER BY year)) * 100, 2) AS yoy_growth_pct
FROM Revenue;
```

Explanation: The self-join pairs each year's row with the prior year's row (r2.year = r1.year - 1) so the growth percentage can be calculated directly. The LAG() window function achieves the same result more simply by looking at the previous row's value in year order without needing a self-join.

Q168. Write a query to find customers who have purchased every product in a given category (relational division).

Setup: *Tables: Products(id, category), Orders(customer_id, product_id)*

Query:

```
SELECT o.customer_id
FROM Orders o
JOIN Products p ON o.product_id = p.id
WHERE p.category = 'Electronics'
GROUP BY o.customer_id
HAVING COUNT(DISTINCT o.product_id) = (
  SELECT COUNT(*) FROM Products WHERE category = 'Electronics'
);
```

Explanation: This is a 'relational division' problem: each customer's distinct purchased products within the category are counted and compared to the total number of products that exist in that category. Only

customers whose distinct purchase count matches the full category count have bought every product in it.

Q169. Write a query to detect orphaned rows — order items referencing a product_id that no longer exists in the Products table.

Setup: *Tables: OrderItems(id, product_id, order_id), Products(id, name)*

Query:

```
SELECT oi.*
FROM OrderItems oi
LEFT JOIN Products p ON oi.product_id = p.id
WHERE p.id IS NULL;
```

Explanation: A LEFT JOIN from OrderItems to Products keeps all order item rows even when no matching product exists; rows where p.id comes back NULL indicate the referenced product_id has no corresponding row in Products, meaning the record is orphaned. This scenario is exactly what a FOREIGN KEY constraint with referential integrity is designed to prevent going forward.

Q170. Write a query to give all employees in the 'Sales' department a 10% raise.

Setup: *Tables: Employees(id, name, salary, dept_id), Departments(id, dept_name)*

Query:

```
UPDATE Employees e
JOIN Departments d ON e.dept_id = d.id
SET e.salary = e.salary * 1.10
WHERE d.dept_name = 'Sales';
```

Explanation: MySQL allows UPDATE statements to include a JOIN, so the Employees table can be joined to Departments to filter by department name, and the SET clause applies a 10% increase only to the rows that satisfy the WHERE condition.

Q171. Write a query to find the median salary in an Employees table.

Setup: *Table: Employees(id, salary)*

Query:

```
SELECT AVG(salary) AS median_salary
FROM (
  SELECT salary,
         ROW_NUMBER() OVER (ORDER BY salary) AS rn,
         COUNT(*) OVER () AS total_count
  FROM Employees
) t
WHERE rn IN (FLOOR((total_count + 1) / 2), CEIL((total_count + 1) / 2));
```

Explanation: The inner query numbers all salaries in sorted order and also records the total row count. The outer query then selects the middle value (for an odd count) or averages the two middle values (for an even count), which is the standard method for computing a median without a dedicated MEDIAN() function in MySQL.

Q172. Write a query to find all employees who share the exact same salary as at least one other employee.

Setup: *Table: Employees(id, name, salary)*

Query:

```
SELECT e.id, e.name, e.salary
FROM Employees e
WHERE e.salary IN (
  SELECT salary FROM Employees GROUP BY salary HAVING COUNT(*) > 1
)
ORDER BY e.salary;
```

Explanation: The subquery groups all employees by salary and uses HAVING COUNT(*) > 1 to find salary values that occur more than once. The outer query then returns every employee whose salary matches one of those duplicated values.

Q173. Write a query to build a simple hierarchy report showing each employee alongside their manager's name.

Setup: *Table: Employees(id, name, manager_id)*

Query:

```
SELECT e.name AS employee_name,
       COALESCE(m.name, 'No Manager (Top Level)') AS manager_name
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.id
ORDER BY manager_name, employee_name;
```

Explanation: This self-join matches each employee to their manager via manager_id = id. A LEFT JOIN (rather than INNER JOIN) is essential here so that top-level employees with no manager (a NULL manager_id) are still included in the results, and COALESCE() substitutes a readable label instead of a blank NULL for those rows.

Q174. Write a query to find, for each department, the employee(s) with the maximum salary — including ties.

Setup: *Table: Employees(id, name, salary, dept_id)*

Query:

```
SELECT e.dept_id, e.name, e.salary
FROM Employees e
WHERE e.salary = (
  SELECT MAX(e2.salary)
  FROM Employees e2
  WHERE e2.dept_id = e.dept_id
);
```

Explanation: The correlated subquery calculates the maximum salary within the same department as the outer row (e2.dept_id = e.dept_id). Because the comparison is an equality check against that maximum rather than a LIMIT 1, every employee who shares the top salary in their department is included, correctly handling ties.

Q175. Write a query to safely archive and remove orders older than 2 years from the live Orders table.

Setup: *Tables: Orders(id, customer_id, order_date, amount), OrdersArchive (same structure)*

Query:

```
START TRANSACTION;
```

```
INSERT INTO OrdersArchive
SELECT * FROM Orders
WHERE order_date < DATE_SUB(CURDATE(), INTERVAL 2 YEAR);

DELETE FROM Orders
WHERE order_date < DATE_SUB(CURDATE(), INTERVAL 2 YEAR);

COMMIT;
```

Explanation: Wrapping both steps in a transaction ensures atomicity: rows are copied into the archive table and only removed from the live table if both operations succeed, and COMMIT finalizes the change. If anything fails in between, a ROLLBACK would restore the original state, preventing data from being deleted without first being safely archived.

17. Quick-Fire Concept Refreshers

Q176. Explain the logical order in which the clauses of a SELECT query are executed.

Answer: Although you write a SELECT statement as SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT, MySQL logically executes it in this order: 1) FROM (and JOINs) — identify and combine source tables, 2) WHERE — filter individual rows, 3) GROUP BY — group the filtered rows, 4) HAVING — filter the groups, 5) SELECT — compute the output expressions/aliases, 6) DISTINCT — remove duplicate rows, 7) ORDER BY — sort the result, 8) LIMIT/OFFSET — restrict the number of rows returned. This is why you can't reference a SELECT alias in a WHERE clause, but you can in ORDER BY — WHERE runs before the alias is created, while ORDER BY runs after.

Q177. What is the difference between UNION and UNION ALL?

Answer: UNION combines the result sets of two or more SELECT queries and automatically removes duplicate rows, which requires an internal sort/comparison step and adds overhead. UNION ALL also combines result sets but keeps all rows including duplicates, making it faster since it skips the deduplication step. UNION ALL should be preferred whenever you know the result sets won't overlap or duplicates are acceptable.

18. Indexing Scenarios

Q178. Your Employee table contains 20 million records and searching by Employee ID is very slow. How can you improve performance?

Setup: *Table: Employee(employee_id, name, department, salary, ...) -- 20 million rows, no index on employee_id*

Query:

```
-- Step 1: Confirm the problem with EXPLAIN
EXPLAIN SELECT * FROM Employee WHERE employee_id = 458213;
-- If 'type' shows ALL and 'rows' is near 20,000,000, it's a full table scan.

-- Step 2: Add an index on the searched column
CREATE INDEX idx_employee_id ON Employee(employee_id);

-- Step 3: Re-check the plan
EXPLAIN SELECT * FROM Employee WHERE employee_id = 458213;
-- 'type' should now show 'ref' or 'const' with 'rows' close to 1.
```

Explanation: The first step is always to confirm the cause with EXPLAIN rather than guessing — a full table scan (type = ALL) on 20 million rows means MySQL is reading every row to find the match. Creating a B-Tree index on employee_id lets MySQL jump almost directly to the matching row using index lookups instead of scanning the whole table, turning an O(n) scan into a near O(log n) lookup. If employee_id is already the PRIMARY KEY, the real issue may instead be an inefficient WHERE clause (e.g., wrapping the column in a function, which disables index use), a missing composite index for a multi-column filter, or table fragmentation — all of which EXPLAIN and ANALYZE TABLE can help diagnose further.

Q179. Why shouldn't you create an index on every column of a table?

Answer: While indexes speed up reads, over-indexing has real costs: every INSERT, UPDATE, and DELETE must update every index on the affected columns, so write performance degrades as index count grows. Each index also consumes additional disk space and memory (since indexes compete with data for space in the InnoDB buffer pool), which can push useful data out of cache and hurt overall performance. Finally, having too many indexes can confuse the query optimizer into choosing a suboptimal execution plan. The right approach is to index selectively — based on actual query patterns — rather than indexing every column by default.

Q180. Which columns in a table should usually be indexed?

Answer: Good candidates for indexing include: primary key and foreign key columns (used constantly in joins), columns frequently used in WHERE clause filters, columns used in JOIN conditions, columns used in ORDER BY or GROUP BY, and columns with high cardinality (many distinct values, like an email or ID) since they narrow down results effectively. Columns that are rarely queried, have very low cardinality (like a boolean 'is_active' flag), or are updated extremely frequently are generally poor candidates, since the maintenance cost of the index can outweigh its benefit.

19. Views & Data Security Scenarios

Q181. HR should be able to see employee salary details, but regular employees should not. How can this be implemented?

Setup: *Table: Employees(id, name, department, email, salary)*

Query:

```
-- View exposing only non-sensitive columns, for general staff access
CREATE VIEW EmployeeDirectory AS
SELECT id, name, department, email
FROM Employees;

-- Grant general staff access only to the view (not the base table)
GRANT SELECT ON company_db.EmployeeDirectory TO 'staff_role'@'%';

-- HR keeps direct access (or a separate view) that includes salary
GRANT SELECT ON company_db.Employees TO 'hr_role'@'%';
```

Explanation: A view can act as a security layer by exposing only a defined subset of columns from the underlying table — here, EmployeeDirectory deliberately omits the salary column. General staff accounts are granted privileges on the view only (and denied direct SELECT on the Employees base table), so they can never query salary data no matter how they phrase their query. HR accounts, by contrast, retain direct privileges on the full Employees table (or a separate HR-specific view), giving them the complete picture. This is a common real-world pattern for column-level access control in MySQL, since it doesn't natively support column-level GRANTs as flexibly as some other databases.

Q182. What is the difference between column-level and row-level security, and how can each be implemented in MySQL?

Answer: Column-level security restricts which columns a user can see (e.g., hiding a salary column), and in MySQL it's typically implemented using a view that excludes sensitive columns, combined with GRANT/REVOKE privileges restricting direct base-table access. Row-level security restricts which rows a user can see (e.g., a manager should only see employees in their own department), and it's typically implemented with a view that includes a WHERE clause filtering rows based on the connected user or a session variable, since MySQL doesn't have built-in native row-level security policies the way some other RDBMSs do. Both approaches rely on layering a view (or an application-level check) between the user and the raw table rather than granting direct table access.

20. Advanced Troubleshooting Scenarios

Q183. A particular query takes 12 seconds to execute. Walk through how you would optimize it.

Answer: First, run EXPLAIN (or EXPLAIN ANALYZE) on the query to see the actual execution plan — check for full table scans (type = ALL), missing index usage, expensive filesorts, or temporary tables. Next, look for common culprits: SELECT * pulling unnecessary columns, functions wrapped around indexed columns in the WHERE clause (which prevents index use, e.g. WHERE YEAR(order_date) = 2025), missing indexes on JOIN or WHERE columns, or a poorly ordered JOIN sequence. Then consider structural fixes: add or adjust indexes (including composite/covering indexes), rewrite correlated subqueries as JOINs, reduce the result set with a narrower WHERE clause or pagination, and check whether the table needs partitioning or archiving of old data. Finally, verify table statistics are current with ANALYZE TABLE, and check server-level factors like buffer pool size, disk I/O, and concurrent load, since a query that's fast in isolation can still be slow under contention.

Q184. Your database suddenly becomes very slow immediately after inserting millions of records. What would you check first?

Answer: First, check whether table statistics are stale by running ANALYZE TABLE, since the query optimizer relies on statistics to choose good execution plans, and a huge bulk insert can leave them badly out of date, causing the optimizer to pick poor plans. Next, check index health and fragmentation — bulk inserts can bloat indexes, so running OPTIMIZE TABLE (for applicable storage engines) may help. Also check the slow query log and EXPLAIN output for previously-fast queries to see if their plans changed, monitor buffer pool hit ratio and disk I/O (since a much larger dataset may no longer fit comfortably in memory), and check for lock contention or replication lag if the insert was still completing or replicating to other servers while queries were running against the table.

Q185. Two users attempt to update the same row at the same time. What problem can occur, and how does MySQL handle it?

Answer: This can lead to a 'lost update' problem, where one transaction's changes silently overwrite another's because both read the same original value before either wrote back their update. MySQL's InnoDB engine handles this primarily through row-level locking: when a transaction updates a row, it acquires an exclusive lock on that row, forcing the second transaction to wait until the first commits or rolls back before it can proceed, preventing them from corrupting each other's changes. The exact behavior also depends on the transaction isolation level in use (e.g., REPEATABLE READ, the InnoDB default) and can be further controlled with explicit locking (SELECT ... FOR UPDATE) or application-level optimistic concurrency control using a version/timestamp column to detect and reject stale updates.

Q186. A DELETE statement accidentally removes thousands of rows from a production table. How can you recover the data?

Answer: If the DELETE was run inside an explicit transaction that hasn't been committed yet, a simple ROLLBACK immediately restores the data — this is why critical DELETE statements should always be tested inside a transaction first. If it was already committed (or autocommit was on, as is MySQL's default), recovery depends on your safety net: restoring from the most recent full backup and then replaying the binary log (binlog) up to just before the accidental DELETE (point-in-time recovery) can reconstruct the lost rows without losing other legitimate changes made afterward. If binlog-based recovery isn't feasible, a replica server that hasn't yet applied the DELETE, or a database snapshot, may also hold the missing data. This scenario is exactly why production systems should always run with binary logging enabled and take regular backups, and why destructive statements should be wrapped in transactions or tested with a SELECT using the same WHERE clause first.

Q187. A customer reports duplicate records in a table. How would you identify and remove them using a modern (MySQL 8.0+) approach?

Setup: *Table: Customers(id, email, name, created_at)*

Query:

```
-- Step 1: Identify duplicates and how many extra copies exist
SELECT email, COUNT(*) AS duplicate_count
FROM Customers
GROUP BY email
HAVING COUNT(*) > 1;

-- Step 2: Use ROW_NUMBER() to tag duplicates, keeping the earliest record
WITH ranked AS (
  SELECT id,
         ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at ASC) AS rn
  FROM Customers
)
DELETE FROM Customers
WHERE id IN (SELECT id FROM ranked WHERE rn > 1);
```

Explanation: The identification step groups rows by the column that should be unique (email) and uses HAVING COUNT(*) > 1 to surface which values have duplicates and how many. For removal, a CTE with ROW_NUMBER() PARTITION BY email numbers each duplicate group starting at 1 for the earliest created_at row; any row numbered greater than 1 is a later duplicate and gets deleted. This window-function approach (available in MySQL 8.0+) is often cleaner to read than a self-join DELETE, and the ORDER BY inside the PARTITION lets you control exactly which copy is kept (e.g., earliest, latest, or lowest ID).